



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE ESTUDIOS SUPERIORES ACATLÁN

Matemáticas Aplicadas y Computación

Bases de Datos Distribuidas

Anexo de Scripts

Proyecto final de bases de datos distribuidas

PRESENTAN

Alcantara Alviso Isaac
Almaraz Mavil Ernesto
Alvarado Martínez Miguel Eduardo
Vargas Nicolas Gabriel

PROFESOR

Rosas Hernández Javier

Scripts del proyecto final ENOE/ENOEN.

03_analizar_estructura_enoe.py

¿Para qué sirve y qué se hizo?

Este script se utilizó para revisar la estructura de los archivos históricos de la ENOE/ENOEN. Su función principal fue recorrer los archivos ZIP disponibles, identificar las familias de datos principales (viv, hog, sdem, coe1 y coe2) y comparar sus encabezados para detectar qué variables se mantenían homogéneas entre periodos. Con este análisis se generó el archivo reporte_columnas_enoe.txt, el cual permitió justificar qué columnas se podían integrar al modelo final y por qué se descartó 2020 1T del proceso de carga histórica.

Código completo del script:

```
import os
import zipfile

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
SALIDA = os.path.join(BASE_DIR, "reporte_columnas_enoe.txt")

FAMILIAS = {
    "VIV": ["viv", "vivienda"],
    "HOG": ["hog", "hogar"],
    "SDEM": ["sdem"],
    "COE1": ["coe1"],
    "COE2": ["coe2"]
}

def leer_encabezado_desde_zip(zip_path, familia_alias):
    try:
        with zipfile.ZipFile(zip_path, 'r') as zf:
            nombres = zf.namelist()
            for nombre in nombres:
                base = os.path.basename(nombre).lower()
                if base.endswith(".csv") and any(alias in base for alias in familia_alias):
                    with zf.open(nombre) as f:
                        primera = f.readline().decode("latin1").strip()
                        return [c.strip() for c in primera.split(",")], nombre
    except Exception:
        return None, None
    return None, None

zip_files = sorted([f for f in os.listdir(BASE_DIR) if f.lower().endswith(".zip")])

lineas = []
lineas.append("REPORTE DE ESTRUCTURA ENOE / ENOEN")
lineas.append("=" * 50)

for familia, alias in FAMILIAS.items():
    lineas.append(f"\nFAMILIA: {familia}")
    lineas.append("-" * 50)
    encontrados = []
    for zip_name in zip_files:
        cols, archivo_interno = leer_encabezado_desde_zip(os.path.join(BASE_DIR, zip_name), alias)
        if cols:
```

```

        encontrados.append((zip_name, archivo_interno, cols))

if not encontrados:
    lineas.append("No se encontraron archivos para esta familia.")
    continue

comunes = set(encontrados[0][2])
todas = set(encontrados[0][2])

for _, _, cols in encontrados[1:]:
    comunes &= set(cols)
    todas |= set(cols)

lineas.append(f"Archivos encontrados: {len(encontrados)}")
lineas.append(f"Columnas comunes: {len(comunes)}")
lineas.append("Listado de columnas comunes:")
for c in sorted(comunes):
    lineas.append(f"  - {c}")

no_comunes = sorted(todas - comunes)
lineas.append(f"\nColumnas no homogéneas: {len(no_comunes)}")
for c in no_comunes:
    lineas.append(f"  - {c}")

with open(SALIDA, "w", encoding="utf-8") as f:
    f.write("\n".join(lineas))

print("Análisis terminado correctamente.")
print("Se generó el archivo:")
print(SALIDA)

```

04_crear_bd_final.py

¿Para qué sirve y qué se hizo?

Este script crea la base de datos centralizada definitiva del proyecto. En él se define la estructura lógica y física del modelo en tercera forma normal, conformado por las tablas periodo, vivienda, hogar y persona. También establece las llaves primarias, las llaves foráneas y las restricciones básicas que garantizan la relación correcta entre las entidades. En otras palabras, aquí se construyó el esquema central sobre el cual después se cargó toda la información histórica.

Código completo del script:

```

import sqlite3
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "enoe_central.db")

if os.path.exists(DB_PATH):
    os.remove(DB_PATH)

conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()

cursor.execute("""

```

```

CREATE TABLE periodo (
    id_periodo INTEGER PRIMARY KEY,
    anio INTEGER NOT NULL,
    trimestre INTEGER NOT NULL,
    UNIQUE(anio, trimestre)
);
""")

cursor.execute("""
CREATE TABLE vivienda (
    id_vivienda INTEGER PRIMARY KEY,
    id_periodo INTEGER NOT NULL,
    cd_a TEXT,
    con TEXT,
    upm TEXT,
    d_sem TEXT,
    n_pro_viv TEXT,
    v_sel TEXT,
    est TEXT,
    t_loc_tri TEXT,
    t_loc_men TEXT,
    tipo TEXT,
    ur TEXT,
    FOREIGN KEY (id_periodo) REFERENCES periodo(id_periodo)
);
""")

cursor.execute("""
CREATE TABLE hogar (
    id_hogar INTEGER PRIMARY KEY,
    id_vivienda INTEGER NOT NULL,
    n_hog TEXT,
    h_mud TEXT,
    d_anio TEXT,
    d_mes TEXT,
    d_dia TEXT,
    p_anio TEXT,
    p_mes TEXT,
    p_dia TEXT,
    r_def TEXT,
    r_pre TEXT,
    FOREIGN KEY (id_vivienda) REFERENCES vivienda(id_vivienda)
);
""")

cursor.execute("""
CREATE TABLE persona (
    id_persona INTEGER PRIMARY KEY,
    id_hogar INTEGER NOT NULL,
    n_ent TEXT,
    per TEXT,
    n_ren TEXT,
    sex TEXT,
    eda TEXT,
    niv_ins TEXT,
    ing7c TEXT,
    ingocup TEXT,
    hrsocup TEXT,
    rama TEXT,

```

```

pos_ocu TEXT,
seg_soc TEXT,
sub_o TEXT,
busqueda TEXT,
clase1 TEXT,
clase2 TEXT,
clase3 TEXT,
emple7c TEXT,
emp_ppal TEXT,
dispo TEXT,
pnea_est TEXT,
zona TEXT,
FOREIGN KEY (id_hogar) REFERENCES hogar(id_hogar),
UNIQUE(id_hogar, n_ent, per, n_ren)
);
"""

conn.commit()
conn.close()

print("Base central final creada correctamente:")
print(DB_PATH)

```

05_cargar_historico_enoe.py

¿Para qué sirve y qué se hizo?

Este script realiza la carga histórica del proyecto. Su trabajo fue abrir cada archivo ZIP válido, identificar automáticamente el año y el trimestre, extraer la información de las familias seleccionadas y poblar las tablas de la base centralizada. Durante este proceso se excluyó 2020 1T y se conservaron los periodos de 2020 3T a 2025 4T. Al finalizar, el script dejó cargados los 22 periodos, así como los registros correspondientes de viviendas, hogares y personas.

Código completo del script:

```

import os
import sqlite3
import zipfile
from io import TextIOWrapper
import pandas as pd

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "enoe_central.db")

def obtener_periodo(nombre_zip):
    nombre = nombre_zip.lower()
    partes = nombre.replace(".zip", "").split("_")
    anio = None
    trimestre = None
    for p in partes:
        if p.isdigit() and len(p) == 4:
            anio = int(p)
        if p.endswith("t") and p[:-1].isdigit():
            trimestre = int(p[:-1])
    return anio, trimestre

```

```

def buscar_csv(zf, clave):
    for nombre in zf.namelist():
        base = os.path.basename(nombre).lower()
        if base.endswith(".csv") and clave in base:
            return nombre
    return None

def leer_csv_desde_zip(zf, nombre_csv):
    with zf.open(nombre_csv) as f:
        return pd.read_csv(TextIOWrapper(f, encoding="latin1"), low_memory=False)

def valor(row, col):
    return row[col] if col in row.index else None

zip_files = sorted([f for f in os.listdir(BASE_DIR) if f.lower().endswith(".zip") and "2020_1t" not in
f.lower()])
print("Archivos ZIP encontrados:", len(zip_files))

conn = sqlite3.connect(DB_PATH)
cursor = conn.cursor()

for zip_name in zip_files:
    anio, trimestre = obtener_periodo(zip_name)
    print(f"\nProcesando {zip_name} -> {anio} T{trimestre}")

    cursor.execute("INSERT OR IGNORE INTO periodo (anio, trimestre) VALUES (?, ?)", (anio, trimestre))
    conn.commit()
    cursor.execute("SELECT id_periodo FROM periodo WHERE anio=? AND trimestre=?", (anio, trimestre))
    id_periodo = cursor.fetchone()[0]

    zip_path = os.path.join(BASE_DIR, zip_name)
    with zipfile.ZipFile(zip_path, 'r') as zf:
        viv_file = buscar_csv(zf, "viv")
        hog_file = buscar_csv(zf, "hog")
        sdem_file = buscar_csv(zf, "sdem")

        df_viv = leer_csv_desde_zip(zf, viv_file)
        df_hog = leer_csv_desde_zip(zf, hog_file)
        df_sdem = leer_csv_desde_zip(zf, sdem_file)

    total_viv = 0
    total_hog = 0
    total_per = 0
    mapa_viv = {}
    mapa_hog = {}

    for _, row in df_viv.iterrows():
        datos = (
            id_periodo, str(valor(row, "cd_a")), str(valor(row, "con")), str(valor(row, "upm")),
            str(valor(row, "d_sem")), str(valor(row, "n_pro_viv")), str(valor(row, "v_sel")),
            str(valor(row, "est")), str(valor(row, "t_loc_tri")), str(valor(row, "t_loc_men")),
            str(valor(row, "tipo")), str(valor(row, "ur"))
        )
        cursor.execute("""
            INSERT INTO vivienda (
                id_periodo, cd_a, con, upm, d_sem, n_pro_viv, v_sel, est,
                t_loc_tri, t_loc_men, tipo, ur
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """, datos)

```

```

id_vivienda = cursor.lastrowid
clave = (str(valor(row, "cd_a")), str(valor(row, "con")), str(valor(row, "upm")),
        str(valor(row, "v_sel"))) if "v_sel" in row.index else str(valor(row, "n_pro_viv"))
mapa_viv[clave] = id_vivienda
total_viv += 1

for _, row in df_hog.iterrows():
    clave_viv = (str(valor(row, "cd_a")), str(valor(row, "con")), str(valor(row, "upm")),
                str(valor(row, "v_sel"))) if "v_sel" in row.index else str(valor(row, "n_pro_viv"))
    id_vivienda = mapa_viv.get(clave_viv)
    if not id_vivienda:
        continue
    datos = (
        id_vivienda, str(valor(row, "n_hog")), str(valor(row, "h_mud")),
        str(valor(row, "d_anio")), str(valor(row, "d_mes")), str(valor(row, "d_dia")),
        str(valor(row, "p_anio")), str(valor(row, "p_mes")), str(valor(row, "p_dia")),
        str(valor(row, "r_def")), str(valor(row, "r_pre"))
    )
    cursor.execute("""
        INSERT INTO hogar (
            id_vivienda, n_hog, h_mud, d_anio, d_mes, d_dia,
            p_anio, p_mes, p_dia, r_def, r_pre
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, datos)
    id_hogar = cursor.lastrowid
    clave_hog = clave_viv + (str(valor(row, "n_hog")),)
    mapa_hog[clave_hog] = id_hogar
    total_hog += 1

for _, row in df_sdem.iterrows():
    clave_viv = (str(valor(row, "cd_a")), str(valor(row, "con")), str(valor(row, "upm")),
                str(valor(row, "v_sel"))) if "v_sel" in row.index else str(valor(row, "n_pro_viv"))
    clave_hog = clave_viv + (str(valor(row, "n_hog")),)
    id_hogar = mapa_hog.get(clave_hog)
    if not id_hogar:
        continue
    datos = (
        id_hogar, str(valor(row, "n_ent")), str(valor(row, "per")), str(valor(row, "n_ren")),
        str(valor(row, "sex")), str(valor(row, "eda")), str(valor(row, "niv_ins")),
        str(valor(row, "ing7c")), str(valor(row, "ingocup")), str(valor(row, "hrsocup")),
        str(valor(row, "rama")), str(valor(row, "pos_ocu")), str(valor(row, "seg_soc")),
        str(valor(row, "sub_o")), str(valor(row, "busqueda")), str(valor(row, "clase1")),
        str(valor(row, "clase2")), str(valor(row, "clase3")), str(valor(row, "emple7c")),
        str(valor(row, "emp_ppal")), str(valor(row, "dispo")), str(valor(row, "pnea_est")),
        str(valor(row, "zona"))
    )
    cursor.execute("""
        INSERT OR IGNORE INTO persona (
            id_hogar, n_ent, per, n_ren, sex, eda, niv_ins, ing7c, ingocup,
            hrsocup, rama, pos_ocu, seg_soc, sub_o, busqueda, clase1, clase2,
            clase3, emple7c, emp_ppal, dispo, pnea_est, zona
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, datos)
    total_per += 1

conn.commit()
print(" Viviendas procesadas:", total_viv)
print(" Hogares procesados: ", total_hog)
print(" Personas procesadas: ", total_per)

```

```

print("\nResumen final de carga:")
for tabla in ["periodo", "vivienda", "hogar", "persona"]:
    cursor.execute(f"SELECT COUNT(*) FROM {tabla}")
    print(f"{tabla.capitalize()}s cargadas:", cursor.fetchone()[0])

conn.close()
print("\nCarga histórica completada correctamente.")

```

06_consultas_centralizadas.py

¿Para qué sirve y qué se hizo?

Este script se desarrolló para ejecutar las cinco consultas principales directamente sobre la base centralizada. Su propósito fue demostrar que la base ya cargada respondía correctamente a preguntas de análisis histórico. Aquí se obtuvieron resultados como el total de personas por periodo, el total por sexo, el promedio de edad, la distribución por nivel educativo y el promedio de ingreso y horas ocupadas. Estas consultas sirvieron como punto de referencia para después compararlas con el esquema distribuido.

Código completo del script:

```

import sqlite3
import os
import pandas as pd

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "enoe_central.db")
conn = sqlite3.connect(DB_PATH)

consulta_1 = """
SELECT p.anio, p.trimestre, COUNT(pe.id_persona) AS total_personas
FROM persona pe
JOIN hogar h ON pe.id_hogar = h.id_hogar
JOIN vivienda v ON h.id_vivienda = v.id_vivienda
JOIN periodo p ON v.id_periodo = p.id_periodo
GROUP BY p.anio, p.trimestre
ORDER BY p.anio, p.trimestre;
"""

consulta_2 = """
SELECT p.anio, p.trimestre, pe.sex, COUNT(pe.id_persona) AS total_personas
FROM persona pe
JOIN hogar h ON pe.id_hogar = h.id_hogar
JOIN vivienda v ON h.id_vivienda = v.id_vivienda
JOIN periodo p ON v.id_periodo = p.id_periodo
GROUP BY p.anio, p.trimestre, pe.sex
ORDER BY p.anio, p.trimestre, pe.sex;
"""

consulta_3 = """
SELECT p.anio, p.trimestre, pe.sex,
       ROUND(AVG(CAST(NULLIF(pe.eda, '') AS REAL)), 2) AS edad_promedio
FROM persona pe
JOIN hogar h ON pe.id_hogar = h.id_hogar
JOIN vivienda v ON h.id_vivienda = v.id_vivienda
JOIN periodo p ON v.id_periodo = p.id_periodo

```



```

WHERE pe.eda IS NOT NULL AND TRIM(pe.eda) <> ''
GROUP BY p.anio, p.trimestre, pe.sex
ORDER BY p.anio, p.trimestre, pe.sex;
"""

consulta_4 = """
SELECT p.anio, p.trimestre, pe.niv_ins, COUNT(pe.id_persona) AS total_personas
FROM persona pe
JOIN hogar h ON pe.id_hogar = h.id_hogar
JOIN vivienda v ON h.id_vivienda = v.id_vivienda
JOIN periodo p ON v.id_periodo = p.id_periodo
GROUP BY p.anio, p.trimestre, pe.niv_ins
ORDER BY p.anio, p.trimestre, total_personas DESC;
"""

consulta_5 = """
SELECT p.anio, p.trimestre,
       ROUND(AVG(CAST(NULLIF(pe.ingocup, '') AS REAL)), 2) AS ingreso_promedio,
       ROUND(AVG(CAST(NULLIF(pe.hrsocup, '') AS REAL)), 2) AS horas_promedio
FROM persona pe
JOIN hogar h ON pe.id_hogar = h.id_hogar
JOIN vivienda v ON h.id_vivienda = v.id_vivienda
JOIN periodo p ON v.id_periodo = p.id_periodo
GROUP BY p.anio, p.trimestre
ORDER BY p.anio, p.trimestre;
"""

print("\nCONSULTA 1: Total de personas por año y trimestre")
print(pd.read_sql_query(consulta_1, conn))

print("\nCONSULTA 2: Total de personas por sexo y periodo")
print(pd.read_sql_query(consulta_2, conn))

print("\nCONSULTA 3: Promedio de edad por sexo y periodo")
print(pd.read_sql_query(consulta_3, conn))

print("\nCONSULTA 4: Distribución por nivel educativo y periodo")
print(pd.read_sql_query(consulta_4, conn))

print("\nCONSULTA 5: Promedio de ingreso y horas ocupadas por periodo")
print(pd.read_sql_query(consulta_5, conn))

conn.close()

```

07_fragmentar_enoe.py

¿Para qué sirve y qué se hizo?

Este script implementa la fragmentación horizontal del proyecto. A partir de la base centralizada, genera dos nuevas bases distribuidas: enoe_frag_1.db y enoe_frag_2.db. El primer fragmento concentra la información de 2020 a 2022 y el segundo la de 2023 a 2025. En este paso se copió la estructura del modelo central y después se trasladaron los registros que correspondían a cada rango temporal. Con ello se construyó el esquema distribuido que se usaría en las siguientes consultas y validaciones.

Código completo del script:

```

import sqlite3
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

CENTRAL_DB = os.path.join(BASE_DIR, "enoe_central.db")
FRAG1_DB = os.path.join(BASE_DIR, "enoe_frag_1.db")
FRAG2_DB = os.path.join(BASE_DIR, "enoe_frag_2.db")

def crear_estructura(db_path):
    conn = sqlite3.connect(db_path)
    cur = conn.cursor()

    cur.execute("DROP TABLE IF EXISTS persona;")
    cur.execute("DROP TABLE IF EXISTS hogar;")
    cur.execute("DROP TABLE IF EXISTS vivienda;")
    cur.execute("DROP TABLE IF EXISTS periodo;")

    cur.execute("""
CREATE TABLE periodo (
    id_periodo INTEGER PRIMARY KEY,
    anio INTEGER NOT NULL,
    trimestre INTEGER NOT NULL,
    UNIQUE(anio, trimestre)
);
""")

    cur.execute("""
CREATE TABLE vivienda (
    id_vivienda INTEGER PRIMARY KEY,
    id_periodo INTEGER NOT NULL,
    cd_a TEXT,
    con TEXT,
    upm TEXT,
    d_sem TEXT,
    n_pro_viv TEXT,
    v_sel TEXT,
    est TEXT,
    t_loc_tri TEXT,
    t_loc_men TEXT,
    tipo TEXT,
    ur TEXT,
    FOREIGN KEY (id_periodo) REFERENCES periodo(id_periodo)
);
""")

    cur.execute("""
CREATE TABLE hogar (
    id_hogar INTEGER PRIMARY KEY,
    id_vivienda INTEGER NOT NULL,
    n_hog TEXT,
    h_mud TEXT,
    d_anio TEXT,
    d_mes TEXT,
    d_dia TEXT,
    p_anio TEXT,
    p_mes TEXT,
    p_dia TEXT,
    r_def TEXT,

```

```

        r_pre TEXT,
        FOREIGN KEY (id_vivienda) REFERENCES vivienda(id_vivienda)
    );
    """

    cur.execute("""
CREATE TABLE persona (
    id_persona INTEGER PRIMARY KEY,
    id_hogar INTEGER NOT NULL,
    n_ent TEXT,
    per TEXT,
    n_ren TEXT,
    sex TEXT,
    eda TEXT,
    niv_ins TEXT,
    ing7c TEXT,
    ingocup TEXT,
    hrsocup TEXT,
    rama TEXT,
    pos_ocu TEXT,
    seg_soc TEXT,
    sub_o TEXT,
    busqueda TEXT,
    clase1 TEXT,
    clase2 TEXT,
    clase3 TEXT,
    emple7c TEXT,
    emp_ppal TEXT,
    dispo TEXT,
    pnea_est TEXT,
    zona TEXT,
    FOREIGN KEY (id_hogar) REFERENCES hogar(id_hogar)
);
    """)

    conn.commit()
    conn.close()

crear_estructura(FRAG1_DB)
crear_estructura(FRAG2_DB)

conn1 = sqlite3.connect(FRAG1_DB)
cur1 = conn1.cursor()
cur1.execute(f"ATTACH DATABASE '{CENTRAL_DB}' AS central;")

cur1.execute("""
INSERT INTO periodo
SELECT *
FROM central.periodo
WHERE anio BETWEEN 2020 AND 2022;
    """)

cur1.execute("""
INSERT INTO vivienda
SELECT v.*
FROM central.vivienda v
JOIN central.periodo p ON v.id_periodo = p.id_periodo
WHERE p.anio BETWEEN 2020 AND 2022;
    """)

```

```

cur1.execute("""
INSERT INTO hogar
SELECT h.*
FROM central.hogar h
JOIN central.vivienda v ON h.id_vivienda = v.id_vivienda
JOIN central.periodo p ON v.id_periodo = p.id_periodo
WHERE p.anio BETWEEN 2020 AND 2022;
""")

cur1.execute("""
INSERT INTO persona
SELECT pe.*
FROM central.persona pe
JOIN central.hogar h ON pe.id_hogar = h.id_hogar
JOIN central.vivienda v ON h.id_vivienda = v.id_vivienda
JOIN central.periodo p ON v.id_periodo = p.id_periodo
WHERE p.anio BETWEEN 2020 AND 2022;
""")

conn1.commit()

cur1.execute("SELECT COUNT(*) FROM periodo;")
f1_periodos = cur1.fetchone()[0]
cur1.execute("SELECT COUNT(*) FROM vivienda;")
f1_viviendas = cur1.fetchone()[0]
cur1.execute("SELECT COUNT(*) FROM hogar;")
f1_hogares = cur1.fetchone()[0]
cur1.execute("SELECT COUNT(*) FROM persona;")
f1_personas = cur1.fetchone()[0]

cur1.execute("DETACH DATABASE central;")
conn1.commit()
conn1.close()

conn2 = sqlite3.connect(FRAG2_DB)
cur2 = conn2.cursor()
cur2.execute(f"ATTACH DATABASE '{CENTRAL_DB}' AS central;")

cur2.execute("""
INSERT INTO periodo
SELECT *
FROM central.periodo
WHERE anio BETWEEN 2023 AND 2025;
""")

cur2.execute("""
INSERT INTO vivienda
SELECT v.*
FROM central.vivienda v
JOIN central.periodo p ON v.id_periodo = p.id_periodo
WHERE p.anio BETWEEN 2023 AND 2025;
""")

cur2.execute("""
INSERT INTO hogar
SELECT h.*
FROM central.hogar h
JOIN central.vivienda v ON h.id_vivienda = v.id_vivienda

```

```

JOIN central.periodo p ON v.id_periodo = p.id_periodo
WHERE p.anio BETWEEN 2023 AND 2025;
""")

cur2.execute("""
INSERT INTO persona
SELECT pe.*
FROM central.persona pe
JOIN central.hogar h ON pe.id_hogar = h.id_hogar
JOIN central.vivienda v ON h.id_vivienda = v.id_vivienda
JOIN central.periodo p ON v.id_periodo = p.id_periodo
WHERE p.anio BETWEEN 2023 AND 2025;
""")

conn2.commit()

cur2.execute("SELECT COUNT(*) FROM periodo;")
f2_periodos = cur2.fetchone()[0]
cur2.execute("SELECT COUNT(*) FROM vivienda;")
f2_viviendas = cur2.fetchone()[0]
cur2.execute("SELECT COUNT(*) FROM hogar;")
f2_hogares = cur2.fetchone()[0]
cur2.execute("SELECT COUNT(*) FROM persona;")
f2_personas = cur2.fetchone()[0]

cur2.execute("DETACH DATABASE central;")
conn2.commit()
conn2.close()

print("Fragmentación completada correctamente.\n")

print("FRAGMENTO 1 (2020-2022)")
print("Periodos :", f1_periodos)
print("Viviendas:", f1_viviendas)
print("Hogares  :", f1_hogares)
print("Personas :", f1_personas)

print("\nFRAGMENTO 2 (2023-2025)")
print("Periodos :", f2_periodos)
print("Viviendas:", f2_viviendas)
print("Hogares  :", f2_hogares)
print("Personas :", f2_personas)

```

08_consultas_equivalentes.py

¿Para qué sirve y qué se hizo?

Este script adapta las cinco consultas principales al contexto distribuido. Para ello, usa ambos fragmentos y reconstruye los resultados globales mediante operaciones como ATTACH DATABASE y UNION ALL. Su objetivo fue comprobar que el esquema distribuido podía producir exactamente los mismos resultados que la base centralizada cuando se realizaban consultas equivalentes. Gracias a este script se confirmó que la fragmentación no alteró la información analítica del proyecto.

Código completo del script:

```

import sqlite3
import os
import pandas as pd

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
FRAG1_DB = os.path.join(BASE_DIR, "enoe_frag_1.db")
FRAG2_DB = os.path.join(BASE_DIR, "enoe_frag_2.db")

conn = sqlite3.connect(":memory:")
cur = conn.cursor()

cur.execute(f"ATTACH DATABASE '{FRAG1_DB}' AS frag1;")
cur.execute(f"ATTACH DATABASE '{FRAG2_DB}' AS frag2;")

consulta_1 = """
SELECT anio, trimestre, SUM(total_personas) AS total_personas
FROM (
    SELECT p.anio, p.trimestre, COUNT(pe.id_persona) AS total_personas
    FROM frag1.persona pe
    JOIN frag1.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag1.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag1.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio, p.trimestre

    UNION ALL

    SELECT p.anio, p.trimestre, COUNT(pe.id_persona) AS total_personas
    FROM frag2.persona pe
    JOIN frag2.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag2.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag2.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio, p.trimestre
)
GROUP BY anio, trimestre
ORDER BY anio, trimestre;
"""

consulta_2 = """
SELECT anio, trimestre, sex, SUM(total_personas) AS total_personas
FROM (
    SELECT p.anio, p.trimestre, pe.sex, COUNT(pe.id_persona) AS total_personas
    FROM frag1.persona pe
    JOIN frag1.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag1.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag1.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio, p.trimestre, pe.sex

    UNION ALL

    SELECT p.anio, p.trimestre, pe.sex, COUNT(pe.id_persona) AS total_personas
    FROM frag2.persona pe
    JOIN frag2.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag2.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag2.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio, p.trimestre, pe.sex
)
GROUP BY anio, trimestre, sex
ORDER BY anio, trimestre, sex;
"""

```

```

consulta_3 = ""
SELECT anio, trimestre, sex,
       ROUND(SUM(suma_edad) * 1.0 / SUM(total_personas), 2) AS edad_promedio
FROM (
    SELECT p.anio, p.trimestre, pe.sex,
           SUM(CAST(NULLIF(pe.eda, '') AS REAL)) AS suma_edad,
           COUNT(pe.id_persona) AS total_personas
    FROM frag1.persona pe
    JOIN frag1.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag1.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag1.periodo p ON v.id_periodo = p.id_periodo
    WHERE pe.eda IS NOT NULL AND TRIM(pe.eda) <> ''
    GROUP BY p.anio, p.trimestre, pe.sex

    UNION ALL

    SELECT p.anio, p.trimestre, pe.sex,
           SUM(CAST(NULLIF(pe.eda, '') AS REAL)) AS suma_edad,
           COUNT(pe.id_persona) AS total_personas
    FROM frag2.persona pe
    JOIN frag2.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag2.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag2.periodo p ON v.id_periodo = p.id_periodo
    WHERE pe.eda IS NOT NULL AND TRIM(pe.eda) <> ''
    GROUP BY p.anio, p.trimestre, pe.sex
)
GROUP BY anio, trimestre, sex
ORDER BY anio, trimestre, sex;
""

consulta_4 = ""
SELECT anio, trimestre, niv_ins, SUM(total_personas) AS total_personas
FROM (
    SELECT p.anio, p.trimestre, pe.niv_ins, COUNT(pe.id_persona) AS total_personas
    FROM frag1.persona pe
    JOIN frag1.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag1.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag1.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio, p.trimestre, pe.niv_ins

    UNION ALL

    SELECT p.anio, p.trimestre, pe.niv_ins, COUNT(pe.id_persona) AS total_personas
    FROM frag2.persona pe
    JOIN frag2.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag2.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag2.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio, p.trimestre, pe.niv_ins
)
GROUP BY anio, trimestre, niv_ins
ORDER BY anio, trimestre, total_personas DESC;
""

consulta_5 = ""
SELECT anio, trimestre,
       ROUND(SUM(suma_ing) * 1.0 / SUM(total_ing), 2) AS ingreso_promedio,
       ROUND(SUM(suma_hrs) * 1.0 / SUM(total_hrs), 2) AS horas_promedio
FROM (

```

```

SELECT p.anio, p.trimestre,
       SUM(CAST(NULLIF(pe.ingocup, '') AS REAL)) AS suma_ing,
       COUNT(CASE WHEN pe.ingocup IS NOT NULL AND TRIM(pe.ingocup) <> '' THEN 1 END) AS total_ing,
       SUM(CAST(NULLIF(pe.hrsocup, '') AS REAL)) AS suma_hrs,
       COUNT(CASE WHEN pe.hrsocup IS NOT NULL AND TRIM(pe.hrsocup) <> '' THEN 1 END) AS total_hrs
FROM frag1.persona pe
JOIN frag1.hogar h ON pe.id_hogar = h.id_hogar
JOIN frag1.vivienda v ON h.id_vivienda = v.id_vivienda
JOIN frag1.periodo p ON v.id_periodo = p.id_periodo
GROUP BY p.anio, p.trimestre

UNION ALL

SELECT p.anio, p.trimestre,
       SUM(CAST(NULLIF(pe.ingocup, '') AS REAL)) AS suma_ing,
       COUNT(CASE WHEN pe.ingocup IS NOT NULL AND TRIM(pe.ingocup) <> '' THEN 1 END) AS total_ing,
       SUM(CAST(NULLIF(pe.hrsocup, '') AS REAL)) AS suma_hrs,
       COUNT(CASE WHEN pe.hrsocup IS NOT NULL AND TRIM(pe.hrsocup) <> '' THEN 1 END) AS total_hrs
FROM frag2.persona pe
JOIN frag2.hogar h ON pe.id_hogar = h.id_hogar
JOIN frag2.vivienda v ON h.id_vivienda = v.id_vivienda
JOIN frag2.periodo p ON v.id_periodo = p.id_periodo
GROUP BY p.anio, p.trimestre
)
GROUP BY anio, trimestre
ORDER BY anio, trimestre;
"""

print("\nCONSULTA EQUIVALENTE 1: Total de personas por año y trimestre")
print(pd.read_sql_query(consulta_1, conn))

print("\nCONSULTA EQUIVALENTE 2: Total de personas por sexo y periodo")
print(pd.read_sql_query(consulta_2, conn))

print("\nCONSULTA EQUIVALENTE 3: Promedio de edad por sexo y periodo")
print(pd.read_sql_query(consulta_3, conn))

print("\nCONSULTA EQUIVALENTE 4: Distribución por nivel educativo y periodo")
print(pd.read_sql_query(consulta_4, conn))

print("\nCONSULTA EQUIVALENTE 5: Promedio de ingreso y horas ocupadas por periodo")
print(pd.read_sql_query(consulta_5, conn))

conn.close()

```

09_consultas_distribuidas.py

¿Para qué sirve y qué se hizo?

Este script contiene cinco consultas pensadas específicamente para aprovechar la distribución de la información. En lugar de reconstruir siempre toda la base, algunas consultas se ejecutan directamente sobre un solo fragmento y otras comparan resultados entre fragmentos. La intención fue mostrar que el diseño distribuido también ofrece ventajas prácticas cuando el análisis se enfoca en ciertos rangos de años o en subconjuntos particulares del histórico.

Código completo del script:


```

import sqlite3
import os
import pandas as pd

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
FRAG1_DB = os.path.join(BASE_DIR, "enoe_frag_1.db")
FRAG2_DB = os.path.join(BASE_DIR, "enoe_frag_2.db")

conn1 = sqlite3.connect(FRAG1_DB)

consulta_1 = """
SELECT p.anio, p.trimestre, COUNT(pe.id_persona) AS total_personas
FROM persona pe
JOIN hogar h ON pe.id_hogar = h.id_hogar
JOIN vivienda v ON h.id_vivienda = v.id_vivienda
JOIN periodo p ON v.id_periodo = p.id_periodo
GROUP BY p.anio, p.trimestre
ORDER BY p.anio, p.trimestre;
"""

consulta_4 = """
SELECT p.anio, p.trimestre, pe.sex,
       ROUND(AVG(CAST(NULLIF(pe.eda, '') AS REAL)), 2) AS edad_promedio
FROM persona pe
JOIN hogar h ON pe.id_hogar = h.id_hogar
JOIN vivienda v ON h.id_vivienda = v.id_vivienda
JOIN periodo p ON v.id_periodo = p.id_periodo
WHERE pe.eda IS NOT NULL AND TRIM(pe.eda) <> ''
GROUP BY p.anio, p.trimestre, pe.sex
ORDER BY p.anio, p.trimestre, pe.sex;
"""

print("\nCONSULTA DISTRIBUIDA 1: Total de personas por periodo (Fragmento 1)")
print(pd.read_sql_query(consulta_1, conn1))

print("\nCONSULTA DISTRIBUIDA 4: Promedio de edad por sexo (Fragmento 1)")
print(pd.read_sql_query(consulta_4, conn1))

conn1.close()

conn2 = sqlite3.connect(FRAG2_DB)

consulta_2 = """
SELECT p.anio, p.trimestre, COUNT(pe.id_persona) AS total_personas
FROM persona pe
JOIN hogar h ON pe.id_hogar = h.id_hogar
JOIN vivienda v ON h.id_vivienda = v.id_vivienda
JOIN periodo p ON v.id_periodo = p.id_periodo
GROUP BY p.anio, p.trimestre
ORDER BY p.anio, p.trimestre;
"""

consulta_3 = """
SELECT p.anio, p.trimestre, pe.sex,
       ROUND(AVG(CAST(NULLIF(pe.ingocup, '') AS REAL)), 2) AS ingreso_promedio
FROM persona pe
JOIN hogar h ON pe.id_hogar = h.id_hogar
JOIN vivienda v ON h.id_vivienda = v.id_vivienda
JOIN periodo p ON v.id_periodo = p.id_periodo

```

```

WHERE pe.ingocup IS NOT NULL AND TRIM(pe.ingocup) <> ''
GROUP BY p.anio, p.trimestre, pe.sex
ORDER BY p.anio, p.trimestre, pe.sex;
"""

print("\nCONSULTA DISTRIBUIDA 2: Total de personas por periodo (Fragmento 2)")
print(pd.read_sql_query(consulta_2, conn2))

print("\nCONSULTA DISTRIBUIDA 3: Promedio de ingreso por sexo (Fragmento 2)")
print(pd.read_sql_query(consulta_3, conn2))

conn2.close()

conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute(f"ATTACH DATABASE '{FRAG1_DB}' AS frag1;")
cur.execute(f"ATTACH DATABASE '{FRAG2_DB}' AS frag2;")

consulta_5 = """
SELECT anio, SUM(total_personas) AS total_personas
FROM (
    SELECT p.anio, COUNT(pe.id_persona) AS total_personas
    FROM frag1.persona pe
    JOIN frag1.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag1.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag1.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio

    UNION ALL

    SELECT p.anio, COUNT(pe.id_persona) AS total_personas
    FROM frag2.persona pe
    JOIN frag2.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag2.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag2.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio
)
GROUP BY anio
ORDER BY anio;
"""

print("\nCONSULTA DISTRIBUIDA 5: Comparación de total de personas por año")
print(pd.read_sql_query(consulta_5, conn))

conn.close()

```

10_validacion_distribucion.py

¿Para qué sirve y qué se hizo?

Este script se utilizó para validar formalmente la distribución. En él se compararon los conteos de registros de la base central con la suma de los dos fragmentos, se revisaron los periodos presentes en cada base y se verificó que el total de personas por año coincidiera exactamente. El resultado esperado era que la diferencia fuera igual a cero, y eso fue precisamente lo que ocurrió. Por esta razón, este script sirve como evidencia de que la fragmentación se realizó correctamente y sin pérdida de información.

Código completo del script:

```
import sqlite3
import os
import pandas as pd

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
CENTRAL_DB = os.path.join(BASE_DIR, "enoe_central.db")
FRAG1_DB = os.path.join(BASE_DIR, "enoe_frag_1.db")
FRAG2_DB = os.path.join(BASE_DIR, "enoe_frag_2.db")

conn = sqlite3.connect(":memory:")
cur = conn.cursor()

cur.execute(f"ATTACH DATABASE '{CENTRAL_DB}' AS central;")
cur.execute(f"ATTACH DATABASE '{FRAG1_DB}' AS frag1;")
cur.execute(f"ATTACH DATABASE '{FRAG2_DB}' AS frag2;")

consulta_totales = """
SELECT 'periodo' AS tabla,
      (SELECT COUNT(*) FROM central.periodo) AS total_central,
      (SELECT COUNT(*) FROM frag1.periodo) AS total_frag1,
      (SELECT COUNT(*) FROM frag2.periodo) AS total_frag2,
      ((SELECT COUNT(*) FROM frag1.periodo) + (SELECT COUNT(*) FROM frag2.periodo)) AS suma_fragmentos,
      ((SELECT COUNT(*) FROM central.periodo) - ((SELECT COUNT(*) FROM frag1.periodo) + (SELECT
COUNT(*) FROM frag2.periodo))) AS diferencia
UNION ALL
SELECT 'vivienda',
      (SELECT COUNT(*) FROM central.vivienda),
      (SELECT COUNT(*) FROM frag1.vivienda),
      (SELECT COUNT(*) FROM frag2.vivienda),
      ((SELECT COUNT(*) FROM frag1.vivienda) + (SELECT COUNT(*) FROM frag2.vivienda)),
      ((SELECT COUNT(*) FROM central.vivienda) - ((SELECT COUNT(*) FROM frag1.vivienda) + (SELECT
COUNT(*) FROM frag2.vivienda)))
UNION ALL
SELECT 'hogar',
      (SELECT COUNT(*) FROM central.hogar),
      (SELECT COUNT(*) FROM frag1.hogar),
      (SELECT COUNT(*) FROM frag2.hogar),
      ((SELECT COUNT(*) FROM frag1.hogar) + (SELECT COUNT(*) FROM frag2.hogar)),
      ((SELECT COUNT(*) FROM central.hogar) - ((SELECT COUNT(*) FROM frag1.hogar) + (SELECT COUNT(*)
FROM frag2.hogar)))
UNION ALL
SELECT 'persona',
      (SELECT COUNT(*) FROM central.persona),
      (SELECT COUNT(*) FROM frag1.persona),
      (SELECT COUNT(*) FROM frag2.persona),
      ((SELECT COUNT(*) FROM frag1.persona) + (SELECT COUNT(*) FROM frag2.persona)),
      ((SELECT COUNT(*) FROM central.persona) - ((SELECT COUNT(*) FROM frag1.persona) + (SELECT
COUNT(*) FROM frag2.persona)));
"""

print("\nVALIDACIÓN 1: Totales generales")
print(pd.read_sql_query(consulta_totales, conn))

consulta_periodos = """
SELECT 'central' AS origen, anio, trimestre
FROM central.periodo
UNION ALL
```

```

SELECT 'frag1' AS origen, anio, trimestre
FROM frag1.periodo
UNION ALL
SELECT 'frag2' AS origen, anio, trimestre
FROM frag2.periodo
ORDER BY origen, anio, trimestre;
"""

print("\nVALIDACIÓN 2: Periodos contenidos en cada base")
print(pd.read_sql_query(consulta_periodos, conn))

consulta_personas_anio = """
SELECT anio,
       SUM(CASE WHEN origen = 'central' THEN total_personas ELSE 0 END) AS central,
       SUM(CASE WHEN origen = 'frag1' THEN total_personas ELSE 0 END) AS frag1,
       SUM(CASE WHEN origen = 'frag2' THEN total_personas ELSE 0 END) AS frag2,
       SUM(CASE WHEN origen IN ('frag1','frag2') THEN total_personas ELSE 0 END) AS suma_fragmentos,
       SUM(CASE WHEN origen = 'central' THEN total_personas ELSE 0 END) -
       SUM(CASE WHEN origen IN ('frag1','frag2') THEN total_personas ELSE 0 END) AS diferencia
FROM (
    SELECT 'central' AS origen, p.anio, COUNT(pe.id_persona) AS total_personas
    FROM central.persona pe
    JOIN central.hogar h ON pe.id_hogar = h.id_hogar
    JOIN central.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN central.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio

    UNION ALL

    SELECT 'frag1' AS origen, p.anio, COUNT(pe.id_persona) AS total_personas
    FROM frag1.persona pe
    JOIN frag1.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag1.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag1.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio

    UNION ALL

    SELECT 'frag2' AS origen, p.anio, COUNT(pe.id_persona) AS total_personas
    FROM frag2.persona pe
    JOIN frag2.hogar h ON pe.id_hogar = h.id_hogar
    JOIN frag2.vivienda v ON h.id_vivienda = v.id_vivienda
    JOIN frag2.periodo p ON v.id_periodo = p.id_periodo
    GROUP BY p.anio
)
GROUP BY anio
ORDER BY anio;
"""

print("\nVALIDACIÓN 3: Personas por año, central vs fragmentos")
print(pd.read_sql_query(consulta_personas_anio, conn))

conn.close()

```

exportar.py

¿Para qué sirve y qué se hizo?

Este script se utilizó como apoyo para la migración de SQLite a PostgreSQL. Su función fue conectarse a las tres bases generadas originalmente en SQLite, leer las tablas periodo, vivienda, hogar y persona, y exportar su contenido a archivos CSV separados. Estos archivos sirvieron como puente de transferencia para recrear la información en PostgreSQL y así generar respaldos en ese gestor sin modificar la lógica original del proyecto.

Código completo del script:

```
import sqlite3
import pandas as pd
import os

BASES = [
    "enoe_central.db",
    "enoe_frag_1.db",
    "enoe_frag_2.db"
]

TABLAS = ["periodo", "vivienda", "hogar", "persona"]

for base in BASES:
    nombre_base = os.path.splitext(base)[0]
    carpeta_salida = f"csv_{nombre_base}"
    os.makedirs(carpeta_salida, exist_ok=True)

    conn = sqlite3.connect(base)

    for tabla in TABLAS:
        print(f"Exportando {tabla} de {base}...")
        df = pd.read_sql_query(f"SELECT * FROM {tabla}", conn)
        ruta_csv = os.path.join(carpeta_salida, f"{tabla}.csv")
        df.to_csv(ruta_csv, index=False, encoding="utf-8")

    conn.close()

print("Exportación terminada.")
```